1. Main components of the harness

Main components were the scheduling, per-task pipeline, and memory handling (between and within tasks). **Scheduling** handled 5 worker threads (one per task) according to what the dispatcher allows. It also handled the clock that stopped new tasks after 3 hr 40 min. In the **per-task pipeline**, a script grep's the spec, finds tests and commands, and creates the first message. 2 parallel attempts are started, as one candidate has its own git worktree. Candidates run tool loops with a limit of 70 turns or 30 minutes. Build gate compiled, checked syntax, and returned errors. An Expert call compares diff with spec. Candidates are graded once finished, and a pass stops others. Near-misses retry. For **memory handling**, this was handled between and within tasks. Between tasks, the Expert writes 10 lessons which are fed into later system prompts. Within a task, if a conversation is more than 160k chars, past turns are summarized and only 14 recent messages are maintained.

The model tiers served different purposes. **Standard** was used for every agent turn and summarization. **Experts** review the diff (only through one call) and do playbook rewrites. **Starter** is not used in the best run.

2. Optimizations and tradeoffs

Working on failing task vs. failing fast: At one point, around 85% of time was spent on tasks that failed anyway, and retries only turned max 2 misses into solves. Instead, tasks that require a big rework are given up on.

Turns and time limits: Various combinations were tested, but 70 turns / 30 min was most successful. Lower limits passed less tests after hitting the turn limit.

Deterministic code: Recon, build gate, and interface check are scripts so that they do not spend tokens and catch errors before grading, which is slow.

Other configurations: Lean (1 candidate, 35 turns, 12 min cap, no retries), cheap (Starter tier), medium (1 candidate, 70 turns, 20-min, near-miss retries)

3. Pitfalls

Pitfalls included exploring without editing (tasks used many turns with no patch), edit tool failures, waiting for slow candidates between the parallel candidates, and tight limits. To solve tasks, it helped to have the build gate, near-miss retries, and a no-edit nudge. Cost per solve improved with prompt caching and keeping the Expert to 1 review. Time per solve improved through upfront grading, fast-fail retries, and env hints.

I started late on this assignment due to personal difficulties so I was not able to iterate as much as I'd liked, but to improve, I would explore "new tests failed" cases and improve time-per-solve.